Distributed Automation Platform

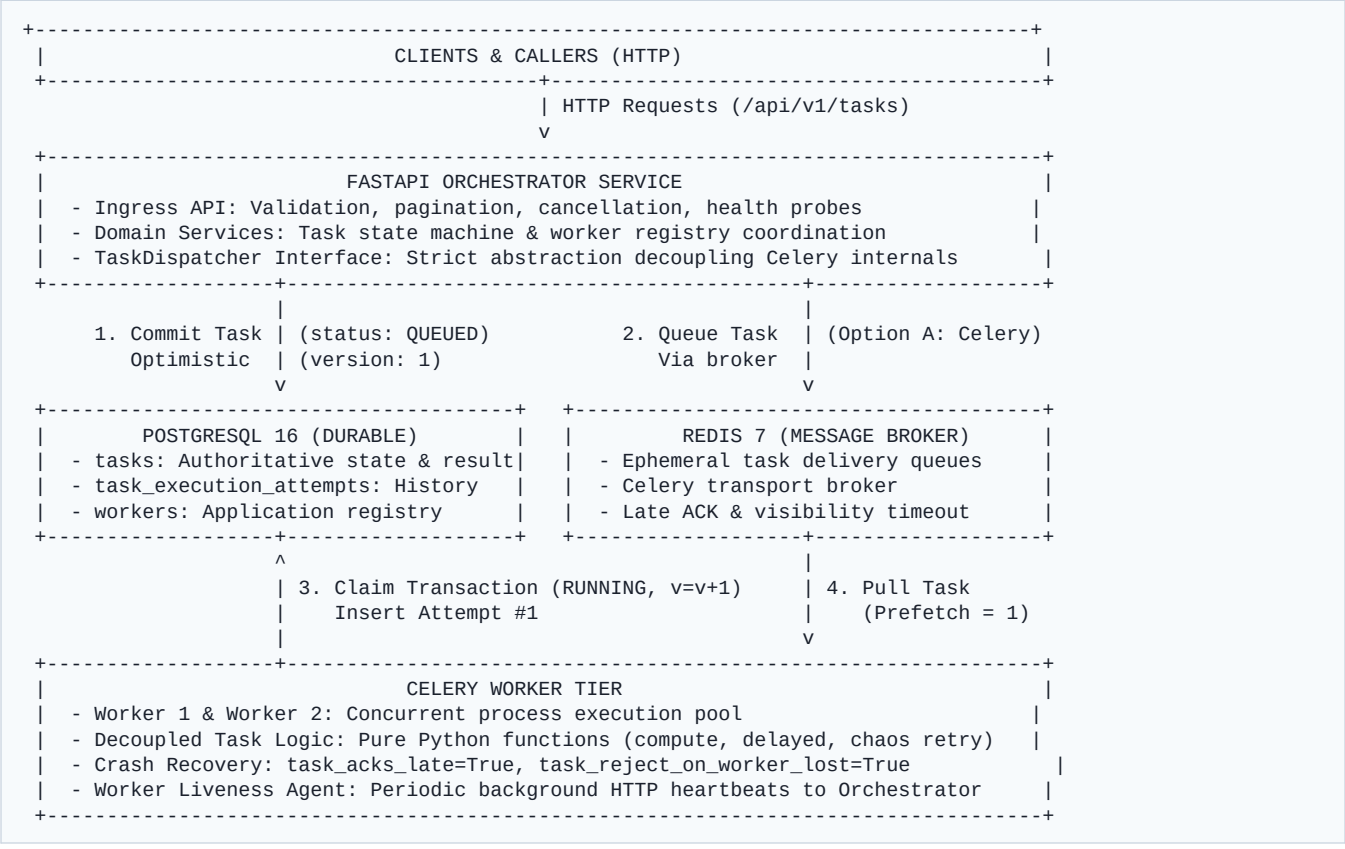
Complete Architecture Blueprint, Implementation Inventory & End-to-End Execution Walkthrough

Runtime: Python 3.12 | FastAPI | Celery 5 | Redis 7 | PostgreSQL 16 | Docker Compose

Project Purpose: *Rebuilt from architectural baseline 01_distributed_automation_platform.tex. This platform solves the problem of slow, failure-prone, and bursty task execution in modern web backends. It decouples synchronous HTTP client interactions from asynchronous, compute-heavy, and distributed background workers. PostgreSQL acts as the single source of truth for durable state and attempt history, Redis acts as the operational broker, and FastAPI acts as the ingress orchestrator.*

1. System Architecture & Topology

The architecture cleanly separates **Ingress & Coordination**, **Operational Messaging**, **Execution**, and **Authoritative Persistence**. Below is the full component layout:



2. Inventory of Implemented Files (Milestone 1)

Every file created in the workspace serves a strictly bounded responsibility without circular dependencies:

File / Module	Subsystem	Exact Architectural Function & Invariants

pyproject.toml	Build & Meta	Defines pinned dependencies (Python 3.12 target, FastAPI, Celery 5.x, Redis, SQLAlchemy 2, Alembic, Pydantic v2, Ruff, Pytest).
.env.example	Config	Complete template of environment settings with production-safe defaults; prevents credentials from entering git.
src/core/config.py	Foundation	Pydantic Settings class parsing environment. Enforces invariant: visibility_timeout > task_time_limit.
src/core/logging.py	Foundation	Structured logging formatter with correlation support, ISO timestamps, and suppression of chatty dependencies.
src/core/exceptions.py	Domain	Domain exception hierarchy: EntityNotFoundError, InvalidStateTransitionError, OptimisticLockError, DispatchError.
src/core/database.py	Persistence	Dual-engine architecture: Async engine/session for FastAPI; Sync engine/session for Celery & Alembic; health check probe.
src/core/redis.py	Broker	Asynchronous Redis connection client with ping probe and graceful connection pool teardown.
src/models/base.py	Persistence	SQLAlchemy 2.0 DeclarativeBase, TimestampMixin (UTC tracking), and VersionMixin (optimistic locking counter).
src/models/task.py	Persistence	TaskRecord (authoritative state, payload, result, version) and TaskExecutionAttempt (audit log with unique attempt constraint).
src/models/worker.py	Persistence	WorkerRecord storing worker ID, hostname, PID, capabilities list, concurrency, and last heartbeat timestamp.
alembic/versions/0001_...	Migrations	DDL script creating tables tasks, task_execution_attempts, workers, foreign keys, and indexes.
src/api/v1/health.py	Ingress API	Liveness probe (/health/live) and readiness probe (/health/ready) returning 503 if DB or Redis is unavailable.
src/main.py	Application	FastAPI factory, lifespan lifecycle manager, and global exception mappers translating domain errors into HTTP 404, 409, and 503.
docker-compose.yml	Deployment	Orchestrates PostgreSQL 16 (with persistent volume), Redis 7, and FastAPI with health checks and service dependencies.

3. End-to-End Execution Dry Run: How Data Travels

Below is the exact execution dry run demonstrating how data flows from HTTP submission to worker execution, outcome persistence, and acknowledgement:

```
[1. Client POST] ----> [2. Validate & Insert QUEUED] ----> [3. Dispatch to Redis Broker]
                            (PostgreSQL, v=1)                      (Celery send_task)
                                                                          |
[6. Late ACK]    <---- [5. Persist Outcome]         <---- [4. Claim & Start Attempt #1]
  (task_acks_late=True)   (status=SUCCEEDED, v=2)              (status=RUNNING, DB lock released)
```

Step 1: Ingress & Validation (FastAPI)

The client sends POST /api/v1/tasks with payload {"task_type": "compute.prime_factorization", "payload": {"n": 104729}}. Pydantic schemas validate types. If validation fails, HTTP 422 is returned immediately with error details.

Step 2: Authoritative Persistence (PostgreSQL First)

The orchestrator persists a new TaskRecord in PostgreSQL inside an ACID transaction: id = UUID, status = 'QUEUED', version = 1, retry_count = 0. **Dual-Write Invariant:** The record is committed to PostgreSQL *before* publishing to Redis. This guarantees no worker can ever process a task that does not exist in the database.

Step 3: Dispatch via Abstraction (Option A)

The orchestrator invokes TaskDispatcher.dispatch(task). CeleryQueueDispatcher routes the message to Redis queue celery. If Redis times out or fails: the task is marked DISPATCH_FAILED and HTTP 503 is returned. **Ambiguous Publish Handling:** If Redis accepted the message before timing out, a worker will still pull it. When the worker detects status DISPATCH_FAILED, its claim reconciliation handler transitions it to RUNNING safely.

Step 4: Worker Pickup & Concurrency-Safe Claim (Phase 1)

An idle Celery worker consumes the message. Before running task logic, it opens a short Claim Transaction in PostgreSQL:
• Executes conditional update: UPDATE tasks SET status='RUNNING', version=version+1 WHERE id=:id AND status='QUEUED' AND version=:v.
• Inserts TaskExecutionAttempt with attempt_number = 1, worker_id = 'worker-1', status = 'RUNNING'.
• Commits the transaction and releases all database locks before executing user code.

Step 5: Pure Task Execution (Phase 2)

The worker executes decoupled Python business logic (e.g. prime factorization). Because database locks were released in Phase 1, long-running compute tasks never exhaust database connection pools.

Step 6: Outcome Persistence (Phase 3)

Upon completion, the worker opens a second transaction:
• Updates task_execution_attempts (status = 'SUCCEEDED', completed_at = NOW(), duration_seconds = 0.042).
• Updates tasks (status = 'SUCCEEDED', result = {...}, version = version + 1).
• Commits the outcome transaction.

Step 7: Late Message Acknowledgement (Phase 4)

Because task_acks_late = True, Celery sends the message ACK to Redis **only after Phase 3 commits**.

4. Distributed Fault Tolerance & System Invariants

- **Worker Crash During Execution (Phase 2):** The Redis message was never acknowledged. Because task_reject_on_worker_lost = True, the supervisor rejects the message upon child crash, and Redis requeues it. Worker 2 claims the task, sees attempt #1 was interrupted, logs attempt #2, and completes the work safely.
- **Worker Crash Between Phase 3 and Phase 4:** The task is committed as SUCCEEDED in PostgreSQL, but the worker dies before ACKing Redis. Redis redelivers the message. Worker 2 inspects PostgreSQL, discovers status == 'SUCCEEDED', immediately sends ACK to Redis, and skips execution (**Idempotency Guard**).
- **The Non-Reassignment Invariant:** A worker heartbeat timeout (marked STALE after 30s) **must NEVER trigger automatic task reassignment by the orchestrator**. A missing heartbeat only reflects telemetry failure. Reassigning tasks based on heartbeats causes split-brain duplicate execution. Redelivery is governed solely by broker acknowledgements.
- **Visibility Timeout & Task Limits:** Redis visibility_timeout (3600s) strictly exceeds task_time_limit (1800s), guaranteeing Redis never redelivers a task while an active worker is still executing it.

5. Verification Evidence & Quality Metrics

1. Automated Tests: 13 tests passed in 0.60s covering config validation, domain exceptions, model creation, unique attempt constraints, and liveness/readiness health probes.
2. Code Standards: 100% compliant with Ruff linting and formatting across all 19 files.
3. Database Schema: Verified via alembic upgrade base:head --sql, generating clean transactional PostgreSQL DDL.
4. Git Versioning: Committed as dc9b2e7 on branch main.